ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA TOÁN - ỨNG DỤNG

—oOo—

BÀI TẬP

HỌC SÂU

ĐỀ TÀI: BÀI TẬP HỒI QUY QUY LOGISTIC VÀ ĐỒ THỊ TÍNH TOÁN

BT02_ComputationGraph

MÔN HỌC: HỌC SÂU

MÃ HỌC PHẦN: 858048

SINH VIÊN THỰC HIỆN: TRƯƠNG TUYẾT NGHI

LỚP: KHOA HỌC DỮ LIỆU - DDU1231

GIẢNG VIÊN: ĐỖ NHƯ TÀI

TP. Hồ Chí Minh, tháng 10, năm 2026

BÀI 1. HỒI QUY LOGISTIC PHÂN LOẠI GIỚI TÍNH TRẺ SƠ SINH

Một cơ sở y tế thu thập dữ liệu về cân nặng và giới tính của một số trẻ sơ sinh. Yêu cầu xây dựng mô hình Logistic Regression để dự đoán giới tính của trẻ dựa trên cân nặng khi sinh.

Tập dữ liệu huấn luyện được cho trong bảng dưới.

STT	Cân nặng x_i (kg)	Giới tính	Nhãn y_i
1	2.4	Nữ	0
2	3.3	Nam	1
3	3.2	Nam	1
4	2.5	Nữ	0

Xét mô hình hồi quy Logistic:

$$\hat{y}_i = h_\theta(x_i) = \sigma(\theta_0 + \theta_1 x_i)$$

Trong đó:

- x_i: cân nặng của trẻ sơ sinh thứ i.
- $y_i \in \{0,1\}$: nhãn giới tính thực tế, với 0 là nữ và 0 là nam.
- $\hat{y}_i$: xác suất mô hình dự đoán trẻ thuộc lớp nam.
- θ_0: hệ số chặn (bias).
- θ_1: trọng số ứng với đặc trưng cân nặng.
- $\sigma(z)$: hàm kích hoạt sigmoid:
$$\sigma(z) = \frac{1}{1+e^{-z}}.$$

Hàm mất mát sử dụng là Binary Cross-Entropy (BCE):

$$J(\theta_0, \theta_1) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \ln(\hat{y}_i) + (1-y_i)\ln(1-\hat{y}_i)\right].$$

Trong đó $m = 4$ là số lượng mẫu huấn luyện.

Sử dụng thuật toán Batch Gradient Descent với các tham số ban đầu:

$$\theta_0^{(0)} = 0.5, \qquad \theta_1^{(0)} = 0.5, \qquad \alpha = 0.2.$$

Quy tắc cập nhật tham số:

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \frac{\partial J}{\partial \theta_j}.$$

Giả sử mỗi lần lặp sử dụng toàn bộ 4 mẫu dữ liệu để tính gradient, các tham số được cập nhật đồng thời và không áp dụng regularization.

a) Viết biểu thức tính đầu vào tuyến tính z_i và xác suất dự đoán $\hat{y}_i$ của mô hình. Xác định giá trị z_i và $\hat{y}_i$ cho từng mẫu dữ liệu với tham số khởi tạo.

Đầu vào tuyến tính:

$$z_i = \theta_0 + \theta_1 x_i.$$

Xác suất dự đoán:

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}.$$

Với $\theta_0^{(0)} = 0.5, \theta_1^{(0)} = 0.5$, ta có:

$$z_i = 0.5 + 0.5x_i.$$

STT	x_i	y_i	z_i	$\hat{y}_i$
1	2.4	0	1.7000	0.845535
2	3.3	1	2.1500	0.895669
3	3.2	1	2.1000	0.890903
4	2.5	0	1.7500	0.851953

b) Tính giá trị Binary Cross-Entropy Loss cho từng mẫu và giá trị hàm mất mát trung bình $J(\theta_0^{(0)}, \theta_1^{(0)})$.

Với từng mẫu:

$$L_i = -\left[y_i \ln(\hat{y}_i) + (1-y_i)\ln(1-\hat{y}_i)\right].$$

Do đó:

$$\begin{aligned}
L_1 &= -\ln(1-0.845535) \approx 1.867786,\\
L_2 &= -\ln(0.895669) \approx 0.110185,\\
L_3 &= -\ln(0.890903) \approx 0.115520,\\
L_4 &= -\ln(1-0.851953) \approx 1.910224.
\end{aligned}$$

Hàm mất mát trung bình:

$$J = \frac{1}{4}(L_1 + L_2 + L_3 + L_4) \approx 1.000929.$$

c) Chứng minh công thức đạo hàm của hàm mất mát theo hai tham số:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_i.$$

Ta có:

Với một mẫu:

$$L_i = -[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)].$$

Ta có:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z)).$$

Suy ra:

$$\frac{\partial L_i}{\partial z_i} = -\left[\frac{y_i}{\hat{y}_i} - \frac{1-y_i}{1-\hat{y}_i}\right]\hat{y}_i(1-\hat{y}_i) = \hat{y}_i - y_i.$$

Mặt khác:

$$z_i = \theta_0 + \theta_1 x_i,$$

nên

$$\frac{\partial z_i}{\partial \theta_0} = 1, \qquad \frac{\partial z_i}{\partial \theta_1} = x_i.$$

Theo quy tắc dây chuyền:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial L_i}{\partial z_i}\frac{\partial z_i}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i),$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial L_i}{\partial z_i}\frac{\partial z_i}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_i.$$

d) Tính giá trị của hai gradient tại thời điểm khởi tạo và thực hiện lần cập nhật Gradient Descent thứ nhất để xác định $\theta_0^{(1)}, \theta_1^{(1)}$.

Tại thời điểm khởi tạo:

$$\hat{y} - y = \begin{bmatrix} 0.845535 \\ -0.104331 \\ -0.109097 \\ 0.851953 \end{bmatrix}.$$

Gradient theo θ_0:

$$\frac{\partial J}{\partial \theta_0} = \frac{0.845535 - 0.104331 - 0.109097 + 0.851953}{4} \approx 0.371015.$$

Gradient theo θ_1:

$$\begin{aligned}\frac{\partial J}{\partial \theta_1} = \frac{1}{4}[&0.845535(2.4) - 0.104331(3.3) \\ &- 0.109097(3.2) + 0.851953(2.5) \quad \approx 0.866441.\end{aligned}$$

Cập nhật:

$$\theta_0^{(1)} = 0.5 - 0.2(0.371015) \approx 0.425797,$$

$$\theta_1^{(1)} = 0.5 - 0.2(0.866441) \approx 0.326712.$$

Vậy:

$$\theta_0^{(1)} \approx 0.425797, \qquad \theta_1^{(1)} \approx 0.326712.$$

e) Sử dụng các tham số sau lần cập nhật thứ nhất, tính lại xác suất dự đoán và gradient trên toàn bộ tập huấn luyện. Thực hiện lần cập nhật thứ hai để xác định $\theta_0^{(2)}, \theta_1^{(2)}$.

Với:

$$\theta_0^{(1)} = 0.425797, \qquad \theta_1^{(1)} = 0.326712,$$

ta tính:

$$z_i = 0.425797 + 0.326712 x_i.$$

Các xác suất dự đoán xấp xỉ:

$$\hat{y} = (0.8274,\ 0.8901,\ 0.8868,\ 0.8330).$$

Từ đó:

$$\frac{\partial J}{\partial \theta_0} \approx 0.294427, \qquad \frac{\partial J}{\partial \theta_1} \approx 0.647762.$$

Cập nhật lần thứ hai:

$$\theta_0^{(2)} = 0.425797 - 0.2(0.294427) \approx 0.366912,$$

$$\theta_1^{(2)} = 0.326712 - 0.2(0.647762) \approx 0.197160.$$

Vậy:

$$\theta_0^{(2)} \approx 0.366912, \qquad \theta_1^{(2)} \approx 0.197160.$$

f) Tính giá trị hàm mất mát trung bình tại ba thời điểm: trước cập nhật, sau lần cập nhật thứ nhất và sau lần cập nhật thứ hai. So sánh và nhận xét sự thay đổi của hàm mất mát.

Các giá trị hàm mất mát là:

$$J^{(0)} \approx 1.000929,$$
$$J^{(1)} \approx 0.843619,$$
$$J^{(2)} \approx 0.758527.$$

Bảng tổng hợp:

Thời điểm	θ_0	θ_1	J
Trước cập nhật	0.500000	0.500000	1.000929
Sau lần 1	0.425797	0.326712	0.843619
Sau lần 2	0.366912	0.197160	0.758527

Hàm mất mát giảm từ 1.000929 xuống 0.843619 rồi xuống 0.758527. Như vậy sau mỗi lần cập nhật, hàm mất mát đều giảm, cho thấy Gradient Descent đang đi theo hướng làm giảm hàm mục tiêu.

g) Sử dụng mô hình sau hai lần cập nhật để dự đoán giới tính của trẻ sơ sinh có cân nặng $x = 2.8$ kg và $x = 3.5$ kg, với ngưỡng phân loại bằng 0.5.

Mô hình sau hai lần cập nhật:

$$\hat{y} = \sigma(0.366912 + 0.197160x).$$

Với $x = 2.8$:

$$z = 0.366912 + 0.197160(2.8) \approx 0.918959,$$
$$\hat{y} \approx 0.714830 > 0.5.$$

Do đó dự đoán:

Nam (nhãn 1)

Với $x = 3.5$:

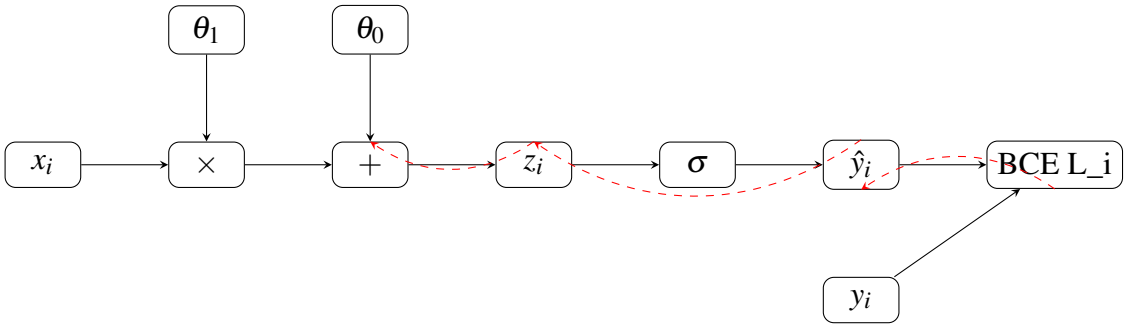
$$z = 0.366912 + 0.197160(3.5) \approx 1.056970,$$
$$\hat{y} \approx 0.742111 > 0.5.$$

Do đó dự đoán:

Nam (nhãn 1)

h) Vẽ đồ thị tính toán (Computation Graph) của mô hình Logistic Regression cho một mẫu huấn luyện, bao gồm các nút tính z, z và BCE Loss. Chỉ rõ chiều lan truyền tiến và chiều lan truyền ngược.

Mũi tên liền biểu diễn lan truyền tiến; mũi tên đứt màu đỏ là chiều lan truyền ngược:



BÀI 2. HỒI QUY LOGISTIC DỰ ĐOÁN ĐÁNH GIÁ PHIM

Một hệ thống đánh giá phim thu thập dữ liệu về thời lượng và mức độ đánh giá của khán giả đối với 14 bộ phim. Mỗi bộ phim được phân thành hai nhóm: Tốt (G – Good) và Không tốt (B – Bad).

Yêu cầu xây dựng mô hình Logistic Regression để dự đoán khả năng một bộ phim được đánh giá tốt dựa trên thời lượng của phim. Tập dữ liệu huấn luyện được cho trong bảng dưới.

Bảng 1: Bảng dữ liệu thời lượng và đánh giá phim

STT	Thời lượng x_i (phút)	Đánh giá	Nhãn y_i
1	132	G	1
2	141	G	1
3	88	B	0
4	154	G	1
5	151	G	1
6	102	B	0
7	119	B	0
8	106	B	0
9	102	B	0
10	143	G	1
11	76	B	0
12	168	G	1
13	94	B	0
14	186	G	1

Xét mô hình hồi quy Logistic:

$$\hat{y}_i = \sigma(wx_i + b).$$

Trong đó:

- x_i: thời lượng của bộ phim thứ i.
- $y_i \in \{0,1\}$: nhãn thực tế, với 1 là phim tốt và 0 là phim không tốt.
- $\hat{y}_i$: xác suất dự đoán phim thuộc nhóm tốt.
- w, b: các tham số cần học.
- $\sigma(z) = \dfrac{1}{1+e^{-z}}$: hàm Sigmoid.

Hàm mất mát sử dụng là Binary Cross-Entropy (BCE):

$$J(w,b) = -\frac{1}{m}\sum_{i=1}^{m}[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)].$$

Với $m = 14$ mẫu dữ liệu. Để thuận tiện cho quá trình huấn luyện, chuẩn hóa đặc trưng thời lượng theo công thức:

$$x_i' = \frac{x_i - 130}{50}.$$

Mô hình khi huấn luyện sử dụng x_i' thay cho x_i. Sử dụng thuật toán Batch Gradient Descent với:

$$w^{(0)} = 0, \qquad b^{(0)} = 0, \qquad \alpha = 0.2.$$

Không sử dụng regularization. Các tham số được cập nhật đồng thời sau mỗi lần tính gradient trên toàn bộ tập huấn luyện.

a) Xác định đầu vào, đầu ra và các tham số cần học của mô hình. Biểu diễn 14 mẫu dữ liệu trên hệ trục tọa độ, với trục hoành là thời lượng phim và trục tung là nhãn đánh giá.

Đầu vào của mô hình là thời lượng phim x_i, khi huấn luyện được thay bằng đặc trưng chuẩn hóa:

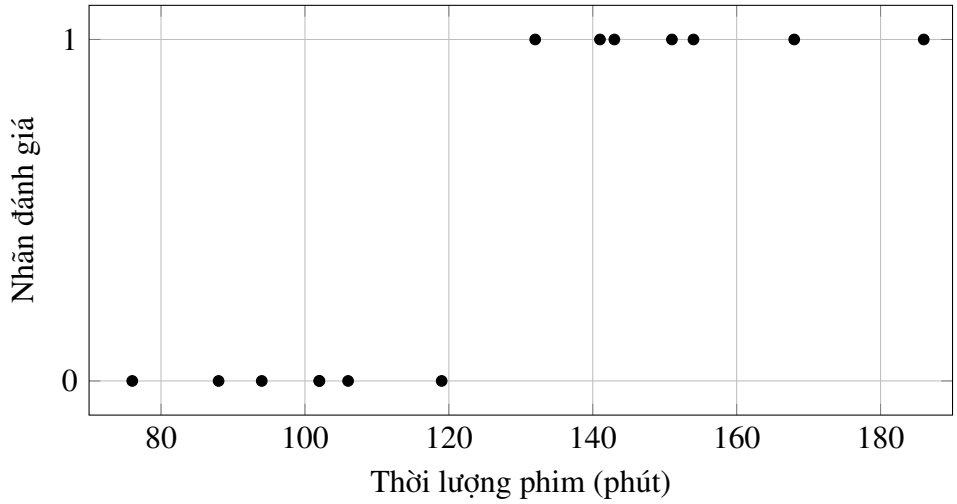
$$x_i' = \frac{x_i - 130}{50}.$$

Đầu ra là xác suất:

$$\hat{y}_i = P(y_i = 1 \mid x_i').$$

Các tham số cần học là w và b.

Biểu diễn dữ liệu trên hệ trục tọa độ:



b) Tính giá trị đặc trưng đã chuẩn hóa x_i', giá trị $z_i = wx_i' + b$ và xác suất dự đoán $\hat{y}_i$ cho từng mẫu tại thời điểm khởi tạo.

Tại thời điểm khởi tạo:

$$w^{(0)} = 0, \qquad b^{(0)} = 0.$$

Vì vậy:

$$z_i = 0 \cdot x_i' + 0 = 0,$$
$$\hat{y}_i = \sigma(0) = 0.5.$$

Bảng tính:

STT	x_i	y_i	x_i'	$z_i, \hat{y}_i$
1	132	1	0.04	0, 0.5
2	141	1	0.22	0, 0.5
3	88	0	-0.84	0, 0.5
4	154	1	0.48	0, 0.5
5	151	1	0.42	0, 0.5
6	102	0	-0.56	0, 0.5
7	119	0	-0.22	0, 0.5
8	106	0	-0.48	0, 0.5
9	102	0	-0.56	0, 0.5
10	143	1	0.26	0, 0.5
11	76	0	-1.08	0, 0.5
12	168	1	0.76	0, 0.5
13	94	0	-0.72	0, 0.5
14	186	1	1.12	0, 0.5

c) Tính Binary Cross-Entropy Loss của từng mẫu và giá trị hàm mất mát trung bình trên toàn bộ tập dữ liệu.

Vì tất cả $\hat{y}_i = 0.5$, với $y_i = 1$:

$$L_i = -\ln(0.5) = 0.693147.$$

Với $y_i = 0$:

$$L_i = -\ln(1-0.5) = 0.693147.$$

Do đó cả 14 mẫu đều có:

$$L_i = 0.693147.$$

Hàm mất mát trung bình:

$$\boxed{J^{(0)} = 0.693147.}$$

d) Chứng minh biểu thức gradient của hàm mất mát:

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_i'$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i).$$

Với: $z_i = wx_i' + b, \qquad \hat{y}_i = \sigma(z_i),$

BCE: $L_i = -[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)].$

Ta có:

$$\frac{\partial L_i}{\partial z_i} = \hat{y}_i - y_i.$$

Đồng thời:

$$\frac{\partial z_i}{\partial w} = x_i', \qquad \frac{\partial z_i}{\partial b} = 1.$$

Theo quy tắc dây chuyền:

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_i',$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i).$$

e) Thực hiện hai lần cập nhật Batch Gradient Descent để xác định $w^{(1)}, b^{(1)}$ và $w^{(2)}, b^{(2)}$. Tính giá trị hàm mất mát sau mỗi lần cập nhật.

Tại lần khởi tạo:

$$\hat{y}_i = 0.5.$$

Ta có:

$$\frac{\partial J}{\partial b} = \frac{1}{14}\sum_{i=1}^{14}(0.5 - y_i) = 0$$

vì có 7 mẫu nhãn 1 và 7 mẫu nhãn 0.

Với w:

$$\frac{\partial J}{\partial w} = \frac{1}{14}\sum_{i=1}^{14}(0.5 - y_i)x_i' \approx -0.277143.$$

Cập nhật lần thứ nhất:

$$w^{(1)} = 0 - 0.2(-0.277143) \approx 0.055429,$$

$$b^{(1)} = 0 - 0.2(0) = 0.$$

Hàm mất mát:

$$J^{(1)} \approx 0.677941.$$

Sau lần cập nhật thứ nhất:

$$\frac{\partial J}{\partial w} \approx -0.271549, \qquad \frac{\partial J}{\partial b} \approx -0.001148.$$

Cập nhật lần thứ hai:

$$w^{(2)} = 0.055429 - 0.2(-0.271549) \approx 0.109738,$$

$$b^{(2)} = 0 - 0.2(-0.001148) \approx 0.000230.$$

Hàm mất mát:

$$J^{(2)} \approx 0.663341.$$

Vậy:

$$w^{(1)} \approx 0.055429, \quad b^{(1)} = 0$$

$$w^{(2)} \approx 0.109738, \quad b^{(2)} \approx 0.000230.$$

f) Viết phương trình mô hình Logistic Regression sau lần cập nhật thứ hai. Sử dụng mô hình để dự đoán xác suất một bộ phim có thời lượng 125 phút và 160 phút được đánh giá tốt.

Mô hình sau lần cập nhật thứ hai:

$$\hat{y} = \sigma(0.109738x' + 0.000230)$$

với:

$$x' = \frac{x - 130}{50}.$$

Với $x = 125$:

$$x' = \frac{125 - 130}{50} = -0.1.$$

Do đó:

$$z = 0.109738(-0.1) + 0.000230 \approx -0.010744,$$
$$\hat{y} \approx 0.497314.$$

Với $x = 160$:

$$x' = \frac{160 - 130}{50} = 0.6.$$

Do đó:

$$z = 0.109738(0.6) + 0.000230 \approx 0.066073,$$
$$\hat{y} \approx 0.516512.$$

g) Với ngưỡng phân loại 0.5, xác định nhãn dự đoán của 14 bộ phim sau lần cập nhật thứ hai. Lập ma trận nhầm lẫn (Confusion Matrix) và tính Accuracy, Precision, Recall và F1-score.

Sau lần cập nhật thứ hai, các xác suất dự đoán xấp xỉ:

STT	y_i	$\hat{y}_i$	$\hat{y}_i \geq 0.5$
1	1	0.501155	1
2	1	0.506093	1
3	0	0.477029	0
4	1	0.513223	1
5	1	0.511578	1
6	0	0.484699	0
7	0	0.494022	0
8	0	0.486892	0
9	0	0.484699	0
10	1	0.507190	1
11	0	0.470462	0
12	1	0.520896	1
13	0	0.480315	0
14	1	0.530745	1

Do đó:

$$TP = 7, \quad TN = 7, \quad FP = 0, \quad FN = 0.$$

Confusion Matrix:

	Dự đoán 0	Dự đoán 1
Thực tế 0	7	0
Thực tế 1	0	7

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{14}{14} = 1.$$

Precision:

$$Precision = \frac{TP}{TP + FP} = \frac{7}{7} = 1.$$

Recall:

$$Recall = \frac{TP}{TP + FN} = \frac{7}{7} = 1.$$

F1-score:

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall} = 1.$$

Vậy:

$$Accuracy = 1, \quad Precision = 1, \quad Recall = 1, \quad F1 = 1.$$

subsection*h) Xác định phương trình đường ranh giới quyết định (Decision Boundary) của mô hình tại ngưỡng xác suất 0.5. Biểu diễn đường Sigmoid cùng các điểm dữ liệu trên hệ trục tọa độ.

Tại ngưỡng:

$$\hat{y} = 0.5,$$

ta có:

$$\sigma(z) = 0.5 \Longleftrightarrow z = 0.$$

Do đó:

$$0.109738x' + 0.000230 = 0.$$

Suy ra:

$$x' = -\frac{0.000230}{0.109738} \approx -0.002092.$$

Đổi về thời lượng phim:

$$x = 50x' + 130 \approx 129.8954.$$

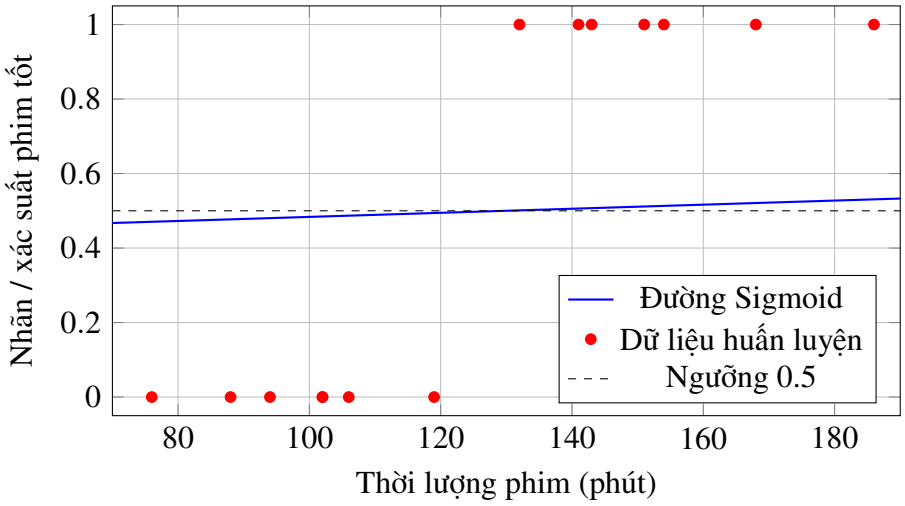
Vậy đường ranh giới quyết định là:

$$x' \approx -0.002092$$

hay trên thang thời lượng ban đầu:

$$x \approx 129.8954 \text{ phút}.$$

Đồ thị biểu diễn xác suất theo thời lượng cùng nhãn dữ liệu:

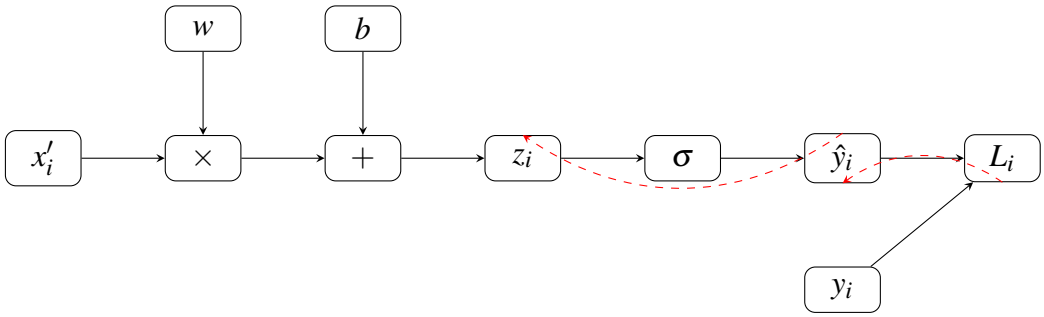


i) Vẽ đồ thị tính toán của mô hình Logistic Regression cho một mẫu dữ liệu. Minh họa quá trình Forward Pass và Backward Pass để tính gradient theo w và b.

Với một mẫu:

$$x' \longrightarrow z = wx' + b \longrightarrow \hat{y} = \sigma(z) \longrightarrow L_{\text{BCE}}.$$

Đồ thị tính toán: Gradient trung bình lấy tổng các đóng góp chia cho 14.



Ta có:

$$\frac{\partial L_i}{\partial z_i} = \hat{y}_i - y_i.$$

Do đó:

$$\frac{\partial L_i}{\partial w} = (\hat{y}_i - y_i)x'_i,$$

$$\frac{\partial L_i}{\partial b} = \hat{y}_i - y_i.$$

j) Viết chương trình Python sử dụng NumPy hoặc PyTorch để huấn luyện mô hình trong 1.000 lần lặp với cùng giá trị khởi tạo và tốc độ học. Vẽ đồ thị hàm mất mát theo số lần lặp và so sánh kết quả với mô hình sau hai lần cập nhật ở câu (e).

Chương trình sử dụng NumPy:

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array(
    [132, 141, 88, 154, 151, 102, 119,
     106, 102, 143, 76, 168, 94, 186],
    dtype=float
)

y = np.array(
    [1, 1, 0, 1, 1, 0, 0,
     0, 0, 1, 0, 1, 0, 1],
    dtype=float
)

x_norm = (x - 130) / 50

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

w = 0.0
b = 0.0
alpha = 0.2
n_iterations = 1000

losses = []

for _ in range(n_iterations):
    z = w * x_norm + b
    y_hat = sigmoid(z)

    eps = 1e-12
    y_hat_clip = np.clip(y_hat, eps, 1 - eps)

    loss = -np.mean(
        y * np.log(y_hat_clip)
        + (1 - y) * np.log(1 - y_hat_clip)
    )
    losses.append(loss)

    dw = np.mean((y_hat - y) * x_norm)
    db = np.mean(y_hat - y)

    w -= alpha * dw
    b -= alpha * db

print("w =", w)
print("b =", b)
print("Loss cuoi =", losses[-1])

plt.plot(range(1, n_iterations + 1), losses)
plt.xlabel("So lan lap")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Loss theo so lan lap")
plt.grid()
plt.show()
```

Sau 1.000 lần lặp:

$$w \approx 7.670320, \qquad b \approx 0.768547.$$

Giá trị loss tại đầu quá trình:

$$J^{(0)} \approx 0.693147.$$

Sau hai lần cập nhật ở câu (e):

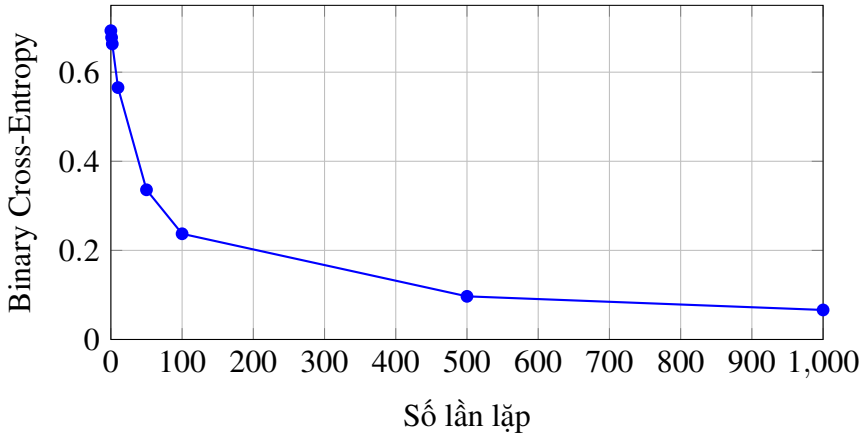
$$J^{(2)} \approx 0.663341.$$

Sau 1.000 lần lặp, loss giảm mạnh, với giá trị cuối xấp xỉ $J \approx 0.066252$.

So với mô hình sau hai lần cập nhật:

$$0.066252 < 0.663341,$$

do đó mô hình sau 1.000 lần lặp có hàm mất mát nhỏ hơn đáng kể.



BÀI TẬP 3. ĐỒ THỊ TÍNH TOÁN VÀ LAN TRUYỀN NGƯỢC

Cho bài toán: Xét hàm số gồm bốn biến đầu vào a, b, c, d, được xác định bởi:

$$J = 3a + 2(4b + 5cd).$$

Với các giá trị ban đầu:

$$a = 2, \qquad b = 3, \qquad c = 4, \qquad d = 5.$$

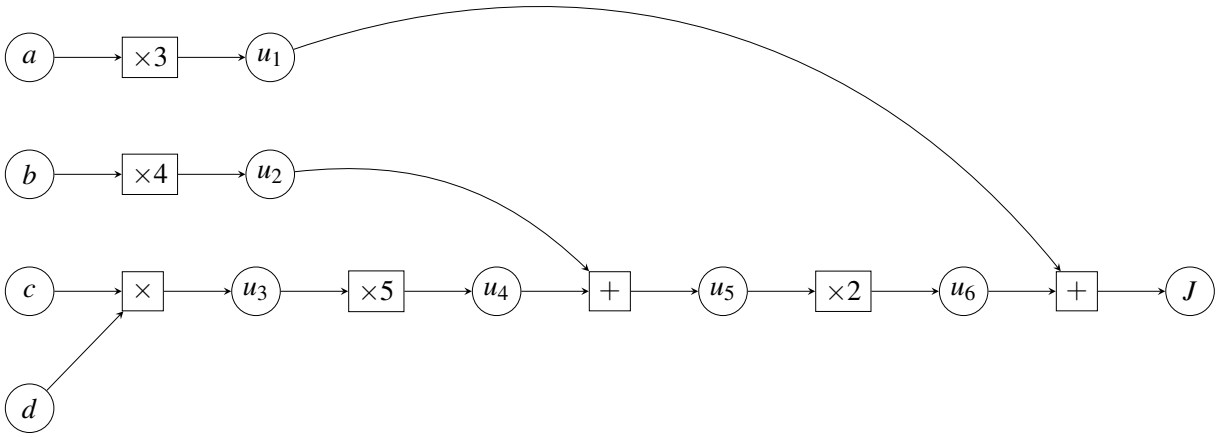
Yêu cầu sử dụng đồ thị tính toán và quy tắc đạo hàm hàm hợp để tính giá trị của hàm số và đạo hàm riêng theo từng biến đầu vào.

a) Phân rã biểu thức J thành các phép toán cơ bản (cộng, nhân) thông qua các biến trung gian:

$$\begin{aligned} u_1 &= 3a, \\ u_2 &= 4b, \\ u_3 &= cd, \\ u_4 &= 5u_3, \\ u_5 &= u_2 + u_4, \\ u_6 &= 2u_5, \\ J &= u_1 + u_6. \end{aligned}$$

b) Vẽ đồ thị tính toán biểu diễn mối quan hệ giữa các biến đầu vào a, b, c, d, các biến trung gian $u_1, \ldots, u_6$ và đầu ra J. Phân biệt rõ nút đầu vào, nút thực hiện phép toán và các cạnh phụ thuộc.

Nút tròn là biến, nút vuông là phép toán, mũi tên biểu diễn sự phụ thuộc.



c) Thực hiện quá trình lan truyền tiến theo đúng thứ tự tính toán của đồ thị. Tính giá trị của tất cả biến trung gian và giá trị đầu ra J.

Với:

$$a = 2, \quad b = 3, \quad c = 4, \quad d = 5,$$

Ta có:

$$u_1 = 3a = 3(2) = 6,$$
$$u_2 = 4b = 4(3) = 12,$$
$$u_3 = cd = 4(5) = 20,$$
$$u_4 = 5u_3 = 5(20) = 100,$$
$$u_5 = u_2 + u_4 = 12 + 100 = 112,$$
$$u_6 = 2u_5 = 2(112) = 224.$$

Cuối cùng:

$$J = u_1 + u_6 = 6 + 224 = 230.$$

Vậy:

$$u_1 = 6, \quad u_2 = 12, \quad u_3 = 20, \quad u_4 = 100, \quad u_5 = 112, \quad u_6 = 224, \quad J = 230.$$

d) Xác định đạo hàm cục bộ tại từng nút tính toán. Viết các biểu thức đạo hàm tương ứng với từng phép cộng và phép nhân.

Với $u_1 = 3a$, ta có:

$$\frac{\partial u_1}{\partial a} = 3.$$

Với $u_2 = 4b$, ta có:

$$\frac{\partial u_2}{\partial b} = 4.$$

Với $u_3 = cd$, ta có:

$$\frac{\partial u_3}{\partial c} = d, \qquad \frac{\partial u_3}{\partial d} = c.$$

Với $u_4 = 5u_3$, ta có:

$$\frac{\partial u_4}{\partial u_3} = 5.$$

Với $u_5 = u_2 + u_4$, ta có:

$$\frac{\partial u_5}{\partial u_2} = 1, \qquad \frac{\partial u_5}{\partial u_4} = 1.$$

Với $u_6 = 2u_5$, ta có:

$$\frac{\partial u_6}{\partial u_5} = 2.$$

Với $J = u_1 + u_6$, ta có:

$$\frac{\partial J}{\partial u_1} = 1, \qquad \frac{\partial J}{\partial u_6} = 1.$$

e) Thực hiện quá trình lan truyền ngược, bắt đầu từ:

$$\frac{\partial J}{\partial J} = 1$$

Áp dụng quy tắc dây chuyền để tính lần lượt gradient của J theo các biến trung gian và xác định:

$$\frac{\partial J}{\partial a}, \quad \frac{\partial J}{\partial b}, \quad \frac{\partial J}{\partial c}, \quad \frac{\partial J}{\partial d}.$$

Ta có:

Bắt đầu:

$$\frac{\partial J}{\partial J} = 1.$$

Tại nút u_1:

$$\frac{\partial J}{\partial u_1} = \frac{\partial J}{\partial J}\frac{\partial J}{\partial u_1} = 1.$$

Tại nút u_6:

$$\frac{\partial J}{\partial u_6} = 1.$$

Tại nút u_5:

$$\frac{\partial J}{\partial u_5} = \frac{\partial J}{\partial u_6}\frac{\partial u_6}{\partial u_5} = 1 \cdot 2 = 2.$$

Tại u_2 và u_4:

$$\frac{\partial J}{\partial u_2} = \frac{\partial J}{\partial u_5}\frac{\partial u_5}{\partial u_2} = 2 \cdot 1 = 2,$$
$$\frac{\partial J}{\partial u_4} = \frac{\partial J}{\partial u_5}\frac{\partial u_5}{\partial u_4} = 2 \cdot 1 = 2.$$

Tại u_3:

$$\frac{\partial J}{\partial u_3} = \frac{\partial J}{\partial u_4}\frac{\partial u_4}{\partial u_3} = 2 \cdot 5 = 10.$$

Do đó:

$$\frac{\partial J}{\partial a} = \frac{\partial J}{\partial u_1}\frac{\partial u_1}{\partial a} = 1 \cdot 3 = 3,$$
$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial u_2}\frac{\partial u_2}{\partial b} = 2 \cdot 4 = 8,$$
$$\frac{\partial J}{\partial c} = \frac{\partial J}{\partial u_3}\frac{\partial u_3}{\partial c} = 10d = 10(5) = 50,$$
$$\frac{\partial J}{\partial d} = \frac{\partial J}{\partial u_3}\frac{\partial u_3}{\partial d} = 10c = 10(4) = 40.$$

Vậy:

$$\frac{\partial J}{\partial a} = 3, \quad \frac{\partial J}{\partial b} = 8, \quad \frac{\partial J}{\partial c} = 50, \quad \frac{\partial J}{\partial d} = 40.$$

f) Kiểm chứng các kết quả gradient ở câu (e) bằng cách khai triển trực tiếp hàm số J và tính đạo hàm riêng theo từng biến.

Khai triển:

$$J = 3a + 2(4b + 5cd) = 3a + 8b + 10cd.$$

Do đó:

$$\frac{\partial J}{\partial a} = 3, \frac{\partial J}{\partial b} = 8,$$
$$\frac{\partial J}{\partial c} = 10d, \frac{\partial J}{\partial d} = 10c.$$

Thay $c = 4, d = 5$:

$$\frac{\partial J}{\partial c} = 10(5) = 50, \frac{\partial J}{\partial d} = 10(4) = 40.$$

Kết quả hoàn toàn trùng với câu (e):

$$\frac{\partial J}{\partial a} = 3, \quad \frac{\partial J}{\partial b} = 8, \quad \frac{\partial J}{\partial c} = 50, \quad \frac{\partial J}{\partial d} = 40.$$

g) Giả sử a, b, c, d là các tham số cần tối ưu và J là hàm mục tiêu cần cực tiểu. Sử dụng thuật toán Gradient Descent với tốc độ học alpha=0.01 để thực hiện một bước cập nhật đồng thời:

$$a' = a - \alpha \frac{\partial J}{\partial a}$$
$$b' = b - \alpha \frac{\partial J}{\partial b}$$
$$c' = c - \alpha \frac{\partial J}{\partial c}$$
$$d' = d - \alpha \frac{\partial J}{\partial d}$$

Tính các giá trị a', b', c', d', sau đó tính lại J' và nhận xét sự thay đổi của hàm mục tiêu.

Với $\alpha = 0.01$ và:

$$\frac{\partial J}{\partial a} = 3, \quad \frac{\partial J}{\partial b} = 8, \quad \frac{\partial J}{\partial c} = 50, \quad \frac{\partial J}{\partial d} = 40,$$

ta có:

$$a' = 2 - 0.01(3) = 1.97,$$
$$b' = 3 - 0.01(8) = 2.92,$$
$$c' = 4 - 0.01(50) = 3.50,$$
$$d' = 5 - 0.01(40) = 4.60.$$

Vậy:

$$a' = 1.97, \quad b' = 2.92, \quad c' = 3.50, \quad d' = 4.60.$$

Tính lại hàm mục tiêu:

$$J' = 3a' + 2(4b' + 5c'd').$$

Thay số:

$$\begin{aligned} J' &= 3(1.97) + 2[4(2.92) + 5(3.5)(4.6)] \\ &= 5.91 + 2[11.68 + 80.5] \\ &= 5.91 + 184.36 \\ &= 190.27. \end{aligned}$$

Giá trị ban đầu:

$$J = 230.$$

Sau cập nhật:

$$J' = 190.27.$$

Do đó:

$$\Delta J = J' - J = 190.27 - 230 = -39.73.$$

Hàm mục tiêu giảm 39.73, cho thấy bước cập nhật Gradient Descent đã làm giảm giá trị hàm mục tiêu.

h) Viết chương trình Python sử dụng PyTorch Autograd để xây dựng lại biểu thức trên, tính J và các gradient thông qua hàm backward(). So sánh kết quả với cách tính thủ công.

```python
import torch

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
c = torch.tensor(4.0, requires_grad=True)
d = torch.tensor(5.0, requires_grad=True)

J = 3*a + 2*(4*b + 5*c*d)

J.backward()

print("J =", J.item())
print("dJ/da =", a.grad.item())
print("dJ/db =", b.grad.item())
print("dJ/dc =", c.grad.item())
print("dJ/dd =", d.grad.item())
```

Kết quả PyTorch:

$$J = 230, \quad \frac{\partial J}{\partial a} = 3, \quad \frac{\partial J}{\partial b} = 8, \quad \frac{\partial J}{\partial c} = 50, \quad \frac{\partial J}{\partial d} = 40.$$

Kết quả này trùng với kết quả tính thủ công ở các câu (e) và (f). nên PyTorch Autograd cho kết quả đúng với phép tính thủ công.